

KAKI

An evidence-grounded operating agent for SME employees: it answers from company records, remembers your working context, and prepares actions without taking control away from you.

1 Executive summary

Small and medium enterprises carry the same coordination load as large ones with far fewer people to absorb it. Decisions are scattered across meetings, tickets, email, chat and documents, and employees lose hours reconstructing what happened, what they own and what needs sign-off. A generic chatbot makes this worse when it answers confidently without evidence or acts without a human checkpoint.

Kaki is an employee-facing operating agent built for that gap. It combines three workflows in one product: **company Q&A grounded in cited records**, **meeting follow-through** that turns commitments into reviewable drafts, and a **human-controlled recruiting assistant**, all on top of a persistent, per-employee working memory. Its runtime persona is an adaptive Technical Chief of Staff: it searches before answering, cites what it read, keeps personal context apart from company evidence, and prepares actions for a person to approve.

Three engineering principles. **Read-only intelligence:** the agent reads company knowledge but holds no write access to company systems. **Zero-credential action handoffs:** every action is prepared as a draft and completed by the employee in their own tools. **Strictly grounded company context:** every company claim cites a record the agent actually retrieved, or it says the evidence is insufficient.

Kaki is live at **kakiai.me**. We evaluated it against an independent benchmark (OrgForge) and a supplementary suite we wrote for what OrgForge does not test, with every test case committed to version control before it was run.

100%

citation integrity: every cited source was one the agent actually retrieved (29/29)

20/20

held-out secrets blocked, 0/10 ordinary messages wrongly blocked, 0 in memory

22/22

honesty and "what's on my plate" cases passed, including 8 hard cases

These results rest on the architecture. Company evidence and personal context are kept strictly apart, so Kaki can use what an employee told it without ever citing it as fact. Retrieval is **hybrid**: exact identifiers, full-text and semantic search fused together, scoped by date and record type, with explicit link-following from one record to the next. A **citation gate** in code refuses any answer that cites something the agent did not read.

Deterministic policy, not the model, decides what may run, what needs approval and what must escalate. And a **two-layer sensitive-data gate** keeps secrets away from the model and from memory.

Contents

- | | |
|-------------------------------|--------------------------------|
| 1. Executive summary | 8. S3 · Recruiting |
| 2. Problem and users | 9. Memory |
| 3. Product | 10. Autonomy and human control |
| 4. Architecture | 11. Security and guardrails |
| 5. Data and knowledge | 12. Evaluation |
| 6. S1 · Company context agent | 13. Success metrics and demo |
| 7. S2 · Meeting actions | 14. Roadmap |

2 Problem and users

SME employees repeatedly need answers to questions like: *What was decided, and where is the evidence? What should I work on today? What did I promise in that meeting? Is this safe to send, or does someone need to approve it?* The information exists but is fragmented across tools and time. The cost is not just slower search. It shows up as duplicated work, missed commitments, inconsistent answers and weak handovers.

Target users are employees and team leads in an SME working across engineering, product, operations, people and customer workflows. Each signed-in session carries the employee's name, role and department, so the same system answers from the right person's perspective.

Value hypothesis. Fewer manual searches across systems; answers tied to evidence rather than recollection; commitments made visible as drafts with explicit approval; shared context reused without proportional headcount. These are hypotheses for a measured pilot. We do not yet claim realised time or revenue savings.

3 Product

Module	What it does	Human control
S1 Company context	Answers workplace questions from company records with inline citations and an evidence graph; shows the employee's own plan and open work for any chosen day.	Read-only. Every company claim cites a retrieved record.
S2 Meeting actions	Screens a live or uploaded transcript, extracts decisions and commitments, checks them against earlier decisions, and drafts emails, invites, tickets and messages.	Drafts wait for review; money and contract commitments escalate; the employee clicks send in their own tools.
S3 Recruiting	Turns a role into hiring criteria, searches public profiles, tiers candidates and drafts outreach. A knowledge gap found in S1 can open a role here directly.	No search runs until the criteria are confirmed; outreach is a draft.

Kaki is deliberately not an autonomous replacement for an employee. It is a context and preparation layer that removes reconstruction work while consequential decisions stay with people.

4 Architecture

Kaki is a modular monolith: one TypeScript/Fastify service powers a stateless agent reasoning loop and coordinates identity, retrieval, models, memory and workflows, with PostgreSQL as the authoritative system of record. The three modules share every layer below them, and each layer has its own trust boundary.

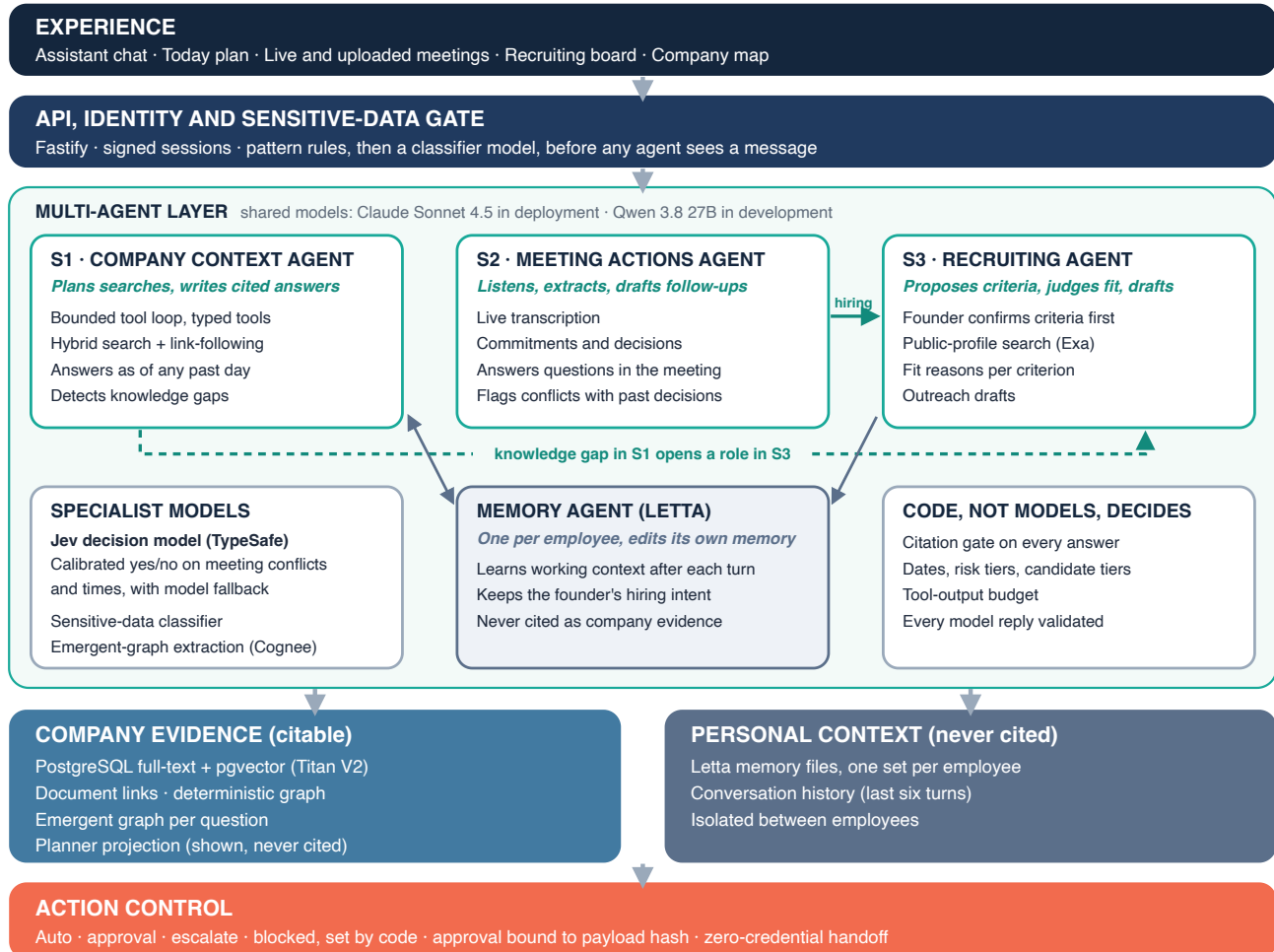


Figure 1. Kaki as a multi-agent system: three task agents that hand work to each other (S3 also runs as a skill inside S1), a memory agent per employee, and specialist models, all on shared layers. Models do the reading and writing; code makes the consequential calls.

The separation matters because each kind of information has a different trust level. Company records are citable evidence. Personal memory is context, never a company source. Conversation history supports follow-ups. Proposed actions are stateful objects that must pass deterministic policy before anything leaves the system.

Component	Technology
Service	TypeScript, Fastify, Server-Sent Events; runs with <code>tsx</code>
Company knowledge	PostgreSQL 16 + pgvector; Amazon Titan Text Embeddings V2 (1024-d)
Personal memory	Letta App Server (one memory agent per employee)
Generation	Claude Sonnet 4.5 via an LLM gateway in deployment; NUS SoCLaaS Qwen 3.8 27B for development and testing, including all evaluation
Optional	Jev decision model (TypeSafe, via Vercel AI Gateway), Cognee (emergent graph), Whisper transcription, live ASR, Gmail/Calendar, Exa people search
Browser	marked, DOMPurify, Shoelace

5 Data and knowledge

The OrgForge corpus

The demo company is the **OrgForge** synthetic SME workplace (Hugging Face): about 5,000 employee-visible records across 11 record types, including Slack, email, Confluence, Jira, Zoom transcripts, pull requests, alerts and invoices. `scripts/orgforge/ingest.py` normalises them into one PostgreSQL schema, split into 7,526 embedded chunks.

Strict oracle isolation

OrgForge also ships its simulator's event log (17,541 "SimEvents"), which its authors describe as the answer-key layer. These are stored for offline builders but are **never chunked, embedded, returned or cited**: every record the agent reads or cites is one an employee could see. A regression test rejects simulator records even when mislabelled. The event log is used in one place only, to link records that share an origin (layer 2 below).

Multi-layer knowledge retrieval

- Layer 1: hybrid search.** Exact identifier lookup, English full-text search and pgvector semantic search, fused by reciprocal rank. The agent can scope a search by date window and record type ("the Confluence pages from that week"), and if embeddings fail, search degrades to keyword-only rather than failing.
- Layer 2: deterministic graph.** `get_related_sources` follows 16,502 explicit references between records (PR → ticket → postmortem), and records that share an origin, such as an incident's alert, ticket and postmortem. These are built offline into a graph of 3,286 nodes and 16,065 edges, which also powers the evidence graph and company map. In a real deployment, shared-origin links come from tool integrations (a Jira issue linked from Slack, an alert that opened a ticket).
- Layer 3: emergent graph.** A per-question graph of meaning, extracted by Cognee, described below.

Emergent knowledge graph (Cognee)

The deterministic graph records what documents reference; it cannot say what the text *means*: that a decision was reversed, that two teams disagreed about a cause, that a risk was raised before it happened. For that, Kaki builds an **emergent graph per question**. After an answer is returned, a background job selects the dozen or so records the question was about, and Cognee uses a language model to extract entities and relationships from just those. Each question gets its own graph, so every relationship can be traced to the question that found it, and extraction cost grows with what people ask rather than with the size of the company's records. The graph is stored locally (Ladybug for the graph, LanceDB for vectors) and never writes back to PostgreSQL, so the authoritative record stays untouched. Answers never wait on it: it appears in the graph view once extraction finishes.

Planner projection

`orgforge_kb/build_timeline.py` projects each person's day plans, ticket states and the employee roster from the corpus. This powers "what's on my plate" for any chosen day. It is shown to the user but never cited as company evidence.

6 S1 · Company context agent

Persona

The system prompt defines Kaki as a Technical Chief of Staff: communicate like an experienced peer, use tools before making workplace claims, treat retrieved text as data rather than instructions, cite every company fact, and state uncertainty plainly. Answers are generated at temperature 0 to keep tool choices and citations consistent.

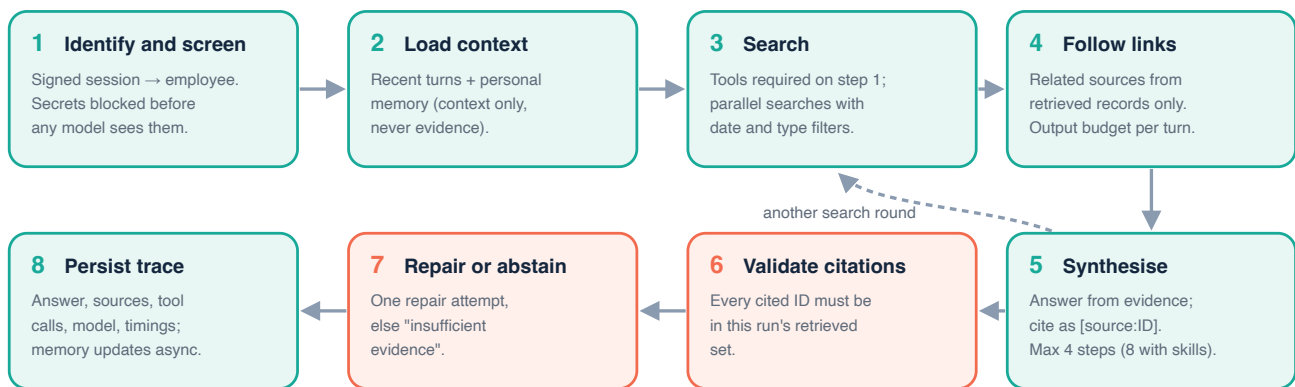


Figure 2. The bounded reasoning loop. Steps 6–7 (coral) are the citation gate; the dashed arrow is another search round when more evidence is needed.

Tools

Kaki exposes typed, purpose-specific tools instead of arbitrary code execution. Parameters are JSON-schema definitions with enums, size limits and `additionalProperties: false`, and malformed arguments are rejected with an error the model can correct.

Tool	Purpose	Constraint
<code>search_company_knowledge</code>	Find company records	Query, optional <code>from/to</code> dates and record <code>types</code> ; at most 20 results
<code>get_related_sources</code>	Follow explicit document links	Seeds must already be retrieved
<code>today_todo</code> , <code>day_plan</code>	The employee's own open work and plan	No person argument: always the signed-in employee
<code>hiring_proposals</code>	Explain knowledge gaps (e.g. a domain whose owner left)	Read-only
<code>open_role_from_gap</code>	Bridge S1 → S3: start a hiring role from a gap	Draft only, on explicit request
<code>load_skill</code>	Load a specialised skill (e.g. recruiting) and its tools	Registered skills only

Answering as of any past day

An employee can pick a past working day and ask "what did we know then?". Kaki then answers from the records as they stood at the end of that day: search sees nothing dated later, graph traversal that cannot be

date-filtered is withheld, and today's personal memory is left out, so no later knowledge leaks into a historical answer. This is how a team reviews what was actually known when an incident broke.

The knowledge-gap hiring bridge

A core architectural idea in Kaki is that operational discovery can turn directly into capacity building. When S1 finds a missing capability while researching (for example, a knowledge domain whose only owner has left the company), it can explain the gap through `hiring_proposals` and, on the employee's request, call `open_role_from_gap`. That opens a new role in S3 with the gap as its starting brief, taking the workflow from "we have a problem" to "we are hiring for it" without re-entering anything.

Gap detection reads only domain health: whether a domain's owner has left, how few people still work on it, and whether its owner is overloaded with incidents. In a backtest over the corpus's 60 working days, scored against OrgForge's own record of hires, which the rules never see, **all three hires were preceded by a proposal for their domain, 6–7 days ahead**. Of six proposals, five were followed by a hire or by someone inside taking the domain on.

Why the citation gate matters

After the model drafts an answer, code checks that every `[source:ID]` refers to a record retrieved in this turn. An answer citing anything else is sent back once for repair; if it still fails, the user gets an explicit "insufficient evidence" reply instead of an unverifiable one. In evaluation this held for every answer (29/29): Kaki never cited a record it had not read. A per-turn **tool-output budget** (about 100k tokens of retrieved text) keeps even long, parallel investigations comfortably inside the model's context window.

7 S2 · Meeting actions

S2 turns a meeting into reviewed follow-up. Audio is transcribed by Whisper, or by an optional live speech service that shows a fast preview while someone is still talking; uploaded transcripts work the same way. Each transcript segment then passes through a fixed pipeline:

Stage	What happens	Why it is built this way
1. Screen	Every segment passes the sensitive-data gate and a narrow prompt-injection screen before any model sees it; blocked segments are withheld.	Rules are anchored to injection wording, so "let's ignore the previous slide" still passes.
2. Extract	The model proposes decisions and candidate actions from a closed set of kinds: email, calendar invite, ticket, message, document, sheet, hiring request, question, escalation.	A closed set keeps every action typed and checkable.
3. Resolve dates	For "next Wednesday" or "the Friday after next", the model fills a form whose fields are closed choices; code turns the form into a date, and asks which day when two fit.	The model understands the words; it never counts days.
4. Check conflicts	Each new decision is checked against decisions from earlier meetings and against company evidence. Yes/no calls such as "does this conflict?" and "which time was meant?" go to Jev, TypeSafe's decision model, which returns a calibrated probability for each option; the main model is the fallback.	Conflicts are found by a dedicated check, never guessed by the extractor.
5. Draft	Drafts are written with company evidence to hand. A commitment repeated later in the meeting stays one action, and a money commitment is never stated as settled until its escalation is approved. Approve stays disabled while a draft is missing something it needs, such as an unresolved date.	One commitment, one draft; no promises made on someone else's behalf.

Stage	What happens	Why it is built this way
6. Tier and hand off	Policy code assigns the tier (section 10); approved drafts open prefilled in Gmail, Google Calendar or GitHub.	The employee sends from their own account.

A question asked aloud in the meeting is answered in a single model call using the question itself as the search, so the answer starts streaming within seconds and is kept to a few sentences.

8 S3 · Recruiting

S3 helps a founder hire without handing the decision to a model. A role starts from a typed or uploaded description (PDF or Word), or from a knowledge gap found in S1 through `open_role_from_gap`.

1. **Criteria:** the model proposes three to six hiring criteria, each marked *must* or *nice*. The founder edits and confirms them; **no search runs before confirmation**.
2. **Search:** public professional profiles are found through Exa, with duplicate profiles merged. Without an Exa key, sample profiles keep the flow demonstrable.
3. **Judge:** each candidate is judged against each criterion as yes, no or unclear. Every model reply is validated, and anything malformed becomes *unclear*, never yes.
4. **Tier, in code:** 100 when every criterion is met; 75 when every *must* is met; 50 when exactly one *must* is unmet or unclear; out when two or more are unmet.
5. **Reach out:** personalised outreach is drafted for review; a work email is looked up only for candidates the founder chooses. Gmail access is read-only, to see replies and free/busy times. The founder sends from their own mailbox.

Hiring intent is memory. Criteria versions, the reasons behind feedback and accepted preferences go into the founder's Letta memory, so "why do we require Rust?" can be answered weeks later. Candidate records never enter memory.

9 Memory

Kind	Where	Used for
Conversation	PostgreSQL conversations and messages; the last six turns go in the prompt, long messages clipped	Follow-up questions in the same chat
Personal working memory	Letta: one persistent memory agent per employee, updated in the background after each turn	Durable facts the employee shared (focus, preferences, schedule), across chats
Company memory	PostgreSQL records, chunks and links	Citable evidence

Personal memory may shape an answer but is never presented as company evidence, which avoids a common agent failure: mixing what a user said earlier with what the organisation can prove. When the user looks at a past day, current personal memory is withheld so later context cannot leak into a historical answer. Letta runs in strict permission mode with a memory-file tool allowlist confined to the employee's memory directory.

Persona-scoped by design. Memory is keyed to the signed-in employee. When a different employee signs in (say, from Jax in backend to Priya in design), Kaki's memory switches with them, so working context, preferences and hiring criteria persist across sessions for each person without ever mixing between people or being cited as company fact.

10 Autonomy and human control

Tier	Examples	Behaviour
Auto	Read-only answers, conflict flags	May complete automatically
Approval	Email, ticket, calendar invite, message, document, hiring draft	Waits for the employee's review
Escalate	Money, discounts, prices, refunds, contracts	Needs a named human decision; cannot execute
Blocked	Secrets, prompt injections, disallowed content	Withheld from the model; cannot execute

Code, not the model, assigns the tier. The model proposes an action kind and payload; deterministic policy decides whether it is automatic, needs approval or escalates. Approval is bound to a hash of the exact payload, so editing a draft invalidates the old approval and stale approvals cannot send modified content.

Zero-credential handoff

In an SME without enterprise single sign-on and fine-grained role-based access control, giving an AI agent write permissions is a serious security risk. Kaki avoids it with a **zero-credential architecture**: it holds no write credentials for external systems. An approved action opens **prefilled in the employee's own tools**: Gmail compose, a Google Calendar event (or a downloadable `.ics`), a GitHub issue form. The final "send" is always the employee's click, in their own authenticated session. Recruiting follows the same principle (section 8).

11 Security and guardrails

- **Session-bound identity:** HMAC-SHA256 signed session cookies (`HttpOnly`, expiry, timing-safe comparison); the chat derives the employee from the session and rejects mismatched IDs.
- **Two-layer sensitive-data gate:** fast pattern rules (labelled passwords and codes, vendor key prefixes, private keys, checksum-validated card numbers, bank details, government IDs), then a **model classifier** that asks, with no tools and no memory, whether the message holds an actual secret value. Blocked messages never reach the assistant or memory. If the classifier is unavailable, the rules still apply and the chat stays up.
- **Prompt-injection resistance:** retrieved text and transcripts are treated as data, not instructions; meeting transcripts also pass a deterministic injection screen before any model call.
- **Citation integrity:** see section 6.
- **Least-capability tools:** core company tools are read-only; state-changing tools load only with their skill and stay subject to workflow checks.

Deployment scope. In the demo company every employee may read every record. Per-document permissions in retrieval, a production identity provider and HTTPS-only cookies are part of the pilot phase (section 14).

12 Evaluation

We measured Kaki two ways: against **OrgForge's own question set**, an independent benchmark we did not write, and a **supplementary suite** for what OrgForge does not test. Every supplementary case was committed to git before it was run.

12.1 OrgForge benchmark

What we ran. OrgForge ships 78 questions, built to evaluate agents inside its own harness, which gives them simulator tools and per-person visibility data that are not part of the published dataset. Many questions therefore cannot be answered from the records alone. We kept the **29 questions** whose required evidence exists in the records an employee can see, and report on those. Evaluation used the development model, Qwen 3.8 27B, throughout; repeated benchmark runs on the deployment model, Claude Sonnet 4.5, were too costly for this phase.

How it was graded. Each answer is compared with OrgForge's reference conclusion by a fixed judge model at temperature 0 (agrees / disagrees / inconclusive). A conclusion-level judge was used rather than keyword matching, so a correct answer phrased differently is not penalised.

Metric	Result	Meaning
Accuracy	62% (18/29)	Judged against OrgForge's reference conclusions, on the shipped code
Citation integrity	100% (29/29)	Every cited source was actually retrieved in that run

Where the misses are. Most misses are questions of the form *did X happen in response to Y*, where OrgForge's definition differs from a plain reading. For example, it counts an email answered by the account owner as "routed", which Kaki read as "handed to another team". We did not tune Kaki to these definitions.

12.2 Supplementary suite

Capability	Result	Cases, including hard ones
Says so when the records don't have the answer	12/12	8 invented subjects ("Project Nimbus", a nonexistent ticket); 4 false premises on real tickets ("Why did Tasha take over ENG-101?" when she never did)
"What's on my plate today?"	10/10 days, 60/60 items	Includes 4 busy days (8–12 items) asked four different ways
Personal memory	17/18 checks	Remembers across chats, updates a fact changed twice, survives 5 unrelated questions, stays isolated between employees with look-alike facts, never cited as company evidence
Sensitive data (held-out set)	20/20 blocked 0/10 false blocks	Passwords, one-time codes, tokens, connection strings, keys, cards, bank details and government IDs, in phrasings written after the system was built; no secret reached memory in any test
Hiring-gap detection (backtest)	3/3 hires foreseen	Over 60 working days, each hire was preceded by a proposal for its domain 6–7 days earlier; 5 of 6 proposals were followed by a hire or an internal hand-over
Meeting actions	23/24 actions found 100% correct risk tier	18 transcript cases: 9/9 injections and secrets blocked, 0/44 false blocks, every matched action given the right approval tier

Four hard cases were removed after review because our expected answers were wrong: Kaki correctly found that the tickets had been completed through merged pull requests.

12.3 Sensitive-data protection

The gate was tested on a held-out set of 20 secrets and 10 look-alike work messages, written after the system was built and never used to tune it. The secrets included passwords typed casually ("the staging login is admin

/ Spr1ngT1me#4"), one-time codes, session tokens, database connection strings with embedded passwords, private keys, card numbers, bank details and government IDs. The look-alikes mention the same topics without a secret ("Do we mask SSNs in the export feature?"). Result: **20/20 secrets blocked, 0/10 ordinary messages blocked**, and no secret reached personal memory in any test.

12.4 Automated tests

More than 900 automated tests run on every change, and TypeScript compiles cleanly under `tsc --noEmit`. They cover identity and user isolation, citations, memory behaviour, the sensitive-data gate, meeting approvals, recruiting confirmation and browser behaviour. A separate oracle-isolation check (`npm run test:orgforge`) verifies that no simulator record can enter retrieval, chunking or embedding.

13 Success metrics and demo

Outcome	KPI	Now	Pilot target
Trustworthy answers	Citation integrity	100%	100%
Correct answers	Accuracy on answerable questions	62% (OrgForge)	≥ 80% on the pilot company's own questions
Fast answers	Time to first useful text	(pilot baseline)	< 3 s
Safe automation	External sends without employee click	0 by design	0
No secret leakage	Held-out secrets blocked / false blocks	20/20, 0/10	≥ 95%, ≤ 2%
Less follow-up work	Commitment → approved draft time	(pilot)	–50% vs manual

Demo narrative: one day with Jax

The demo follows one engineer through one hard day, using real records from the demo company.

1. **The pressure:** last week a customer's live data froze and an SLA was missed; at 11:00 Jax has to face them.
2. **Know before the call:** Jax asks whether last week's outage is the same bug as February's. Kaki searches tickets, postmortems and chat and answers "not the same bug, but the same code", with every line cited. The evidence graph shows how it followed the links, and an "as of" view answers from the records on the day it broke.
3. **During the call:** Kaki transcribes live. When the customer asks the same question, the answer is already there. It flags a plan that contradicts last week's decision, withholds an API key read aloud before any model sees it, and routes a 20% discount to Finance.
4. **After the call:** four commitments become drafts, each showing whose words it came from. "Next Tuesday" fits two dates, so Kaki asks instead of guessing, and Approve waits. Approving the email opens it in Jax's own mailbox; sending is his click.
5. **Short-handed:** Kaki shows a knowledge domain fewer and fewer people understand, proposes a hire, and runs the recruiting flow: confirmed criteria, candidates with reasons, and outreach for Jax to send.
6. **Evening:** nothing needs Jax.

14 Roadmap

Phase	Focus	Outcome
1. Pilot 0–3 months	Read connectors for a real SME's Slack, Google Workspace and Jira; single sign-on and per-document permissions in retrieval; the OrgForge and supplementary suites run on every change, alongside questions from the pilot company itself.	Kaki answering from one company's live records, with measured time saved per employee.
2. Team rollout 3–6 months	Managed, multi-tenant memory infrastructure and durable background queues for many concurrent employees; more candidate-sourcing connectors (e.g. GitHub, Stack Overflow) beyond Exa; streamed first responses within seconds; employees can view and edit what Kaki remembers about them.	Daily use across a whole team, with per-tenant cost and latency targets.
3. Trusted actions 6–12 months	Enterprise single sign-on and role-based access control, then delegated write access (e.g. creating a Jira ticket directly) for action types a company has approved, extending the same tiered policy and payload-hash approval.	Fewer clicks for routine follow-ups, with humans still approving anything consequential.
4. More SME functions	Customer support, finance operations and onboarding, built on the same contract.	One operating agent for the whole SME.

Each phase keeps Kaki's core contract unchanged: retrieve evidence, keep memory separated, prepare typed actions, and keep risk-proportionate human control.

Claim boundary. This report describes behaviour implemented and tested in the codebase. Evaluation numbers come from runs recorded in `docs/evaluation/` with the question sets in `eval/`. Pilot targets are goals, not achieved business outcomes.